

# Space Weather Radio-Risk Monitor

MSDS 682 Data Stream Processing, Summer 2026 — Final Report

Noam Fein (nsfein) and Niki Naderzad (nnaderzad) · August 2026

Repository: [github.com/noamsfein/Space-Weather-Radio-Risk-Monitor](https://github.com/noamsfein/Space-Weather-Radio-Risk-Monitor)

## 1. Problem and useful result

---

Amateur-radio operators depend on high-frequency (HF) propagation, which can degrade during geomagnetic storms. NOAA publishes an estimated planetary K-index (Kp) roughly every minute, but an operator does not want to watch a feed; they want one machine-readable signal when conditions cross into storm territory. Kp 6 corresponds to a G2 geomagnetic storm, a level at which NOAA notes HF propagation can fade at higher latitudes.

The product is deliberately narrow: one data source, one Kafka topic ( `kp_observations` ), one canonical event contract, a rolling 15-minute event-time maximum, and one deterministic alert rule — *emit an alert when the rolling maximum crosses from below 6 to at-or-above 6, and rearm only after it falls back below 6*. It does not forecast, does not model a specific station or location, and never lets AI decide anything about the alert.

The observable result: replaying the project's labeled nine-event fixture through a real local Kafka broker produces exactly two alerts in `outputs/alert.json`. The first is:

```
{
  "alert_id": "kp-alert-20260811T001000Z",
  "triggered_at": "2026-08-11T00:10:00Z",
  "latest_kp": 6.3,
  "rolling_15m_max_kp": 6.3,
  "window_start_utc": "2026-08-10T23:55:00Z",
  "window_end_utc": "2026-08-11T00:10:00Z",
  "window_minutes": 15.0,
  "threshold_kp": 6.0,
  "risk_label": "elevated",
  "source": "synthetic://kp-threshold-fixture"
}
```

Figure 1. First alert from the verified demo run. Every field is rule-derived; there is no AI-derived field.

## 2. Data source and event contract

---

The source is the NOAA/NWS Space Weather Prediction Center public JSON service

`https://services.swpc.noaa.gov/json/planetary_k_index_1m.json` — public HTTPS, no key, no personal data, public-domain NWS terms with attribution. Its documented limitations shaped the design: the values are *estimated and preliminary* (not independent one-minute measurements), they may be revised or missing, planetary Kp is global rather than location-specific, and the feed is often quiet. Dated source checks on both August 11 and August 12, 2026 found 358 well-formed, unique, ordered records at one-minute cadence — and a maximum Kp under 3.5. A quiet live feed cannot

demonstrate a Kp 6 alert, which is why the required review path replays a committed, clearly labeled synthetic threshold fixture instead of live data.

### Raw-to-canonical mapping and validation

A raw record has four fields (`time_tag`, `kp_index`, `estimated_kp`, `kp`). Normalization keeps `time_tag`, renames `estimated_kp` to `kp_value`, and adds provenance and ingestion time. Every event — cached NOAA or replay — must pass the same Pydantic contract (`src/contract.py`): timestamps parse and normalize to UTC (NOAA omits the `Z`; the documented source convention is UTC), `kp_value` is numeric, finite, and in `[0, 9]`, and `source` must be either the NOAA endpoint or the synthetic fixture identifier — anything else is rejected, so synthetic data can never masquerade as NOAA history. The model is frozen and forbids extra fields.

```
{
  "time_tag": "2026-08-11T00:10:00Z",
  "kp_value": 6.3,
  "source": "synthetic://kp-threshold-fixture",
  "ingested_at": "2026-08-11T00:10:05Z"
}
```

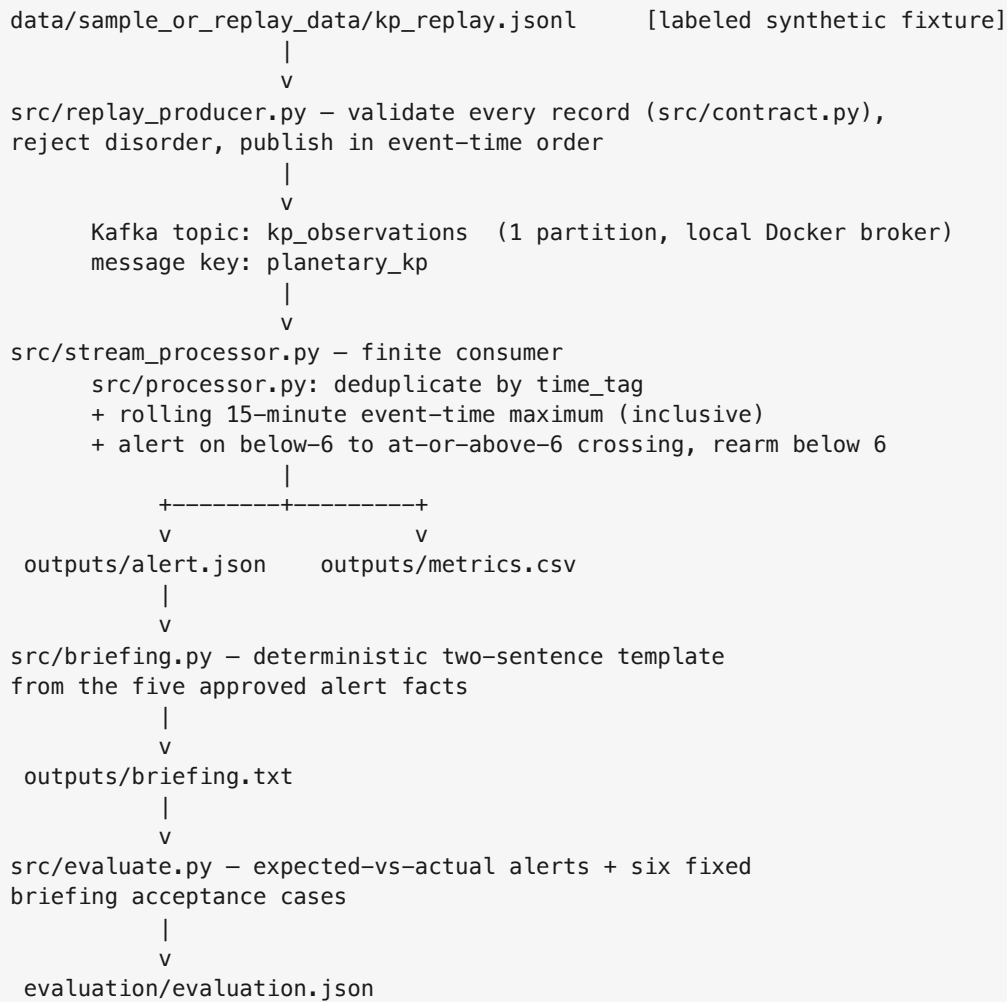
Figure 2. The representative event (`data/fixtures/representative_event.json`) — the first alert-producing record of the replay. The same four field names appear in code, tests, documentation, and slides.

Kafka topic: `kp_observations`, one partition. Message key: the constant `planetary_kp`, because there is exactly one global planetary sequence — a constant key keeps it ordered even if the topic ever gains partitions. Deduplication identity is `time_tag`. The replay fixture deliberately contains one duplicate timestamp so the demo proves deduplication rather than assuming it.

## 3. Architecture and implementation

---

#### REQUIRED REPLAY PATH (no network, no API key)



#### OPTIONAL, OFF THE REQUIRED PATH

- live OpenAI briefing (--use-live-ai): private key in .env; the candidate must pass the same fact checks or the deterministic fallback is used
- live NOAA poller (src/live\_poller.py): fetch once or every 60 seconds, normalize new timestamps, and publish them to the same Kafka topic

Figure 3. Implemented architecture. Every box maps to a submitted module or artifact.

### One real replay example

The table below is the actual `outputs/metrics.csv` from the verified demo run — one row per consumed Kafka message. It shows the whole story: the first crossing at 00:10; the duplicate of that message skipped without re-alerting; still-elevated windows (rolling max 6.3) producing no duplicate alert while new values are below 6; the window decaying (5.5, then below 6 at 00:26, which rearms the alert); and a second crossing at 00:27.

| time_tag             | kp_value | rolling_15m_max_kp | risk_label | alert_emitted | processing_status        |
|----------------------|----------|--------------------|------------|---------------|--------------------------|
| 2026-08-11T00:00:00Z | 4.0      | 4.0                | normal     | false         | accepted                 |
| 2026-08-11T00:05:00Z | 5.0      | 5.0                | normal     | false         | accepted                 |
| 2026-08-11T00:10:00Z | 6.3      | 6.3                | elevated   | <b>true</b>   | accepted                 |
| 2026-08-11T00:10:00Z | 6.3      | 6.3                | elevated   | false         | <b>duplicate_skipped</b> |
| 2026-08-11T00:20:00Z | 5.5      | 6.3                | elevated   | false         | accepted                 |
| 2026-08-11T00:25:00Z | 5.2      | 6.3                | elevated   | false         | accepted                 |
| 2026-08-11T00:26:00Z | 5.0      | 5.5                | normal     | false         | accepted                 |
| 2026-08-11T00:27:00Z | 7.0      | 7.0                | elevated   | <b>true</b>   | accepted                 |
| 2026-08-11T00:30:00Z | 6.7      | 7.0                | elevated   | false         | accepted                 |

Figure 4. Actual metrics rows from the demo run; they match the committed expected-results oracle exactly.

## Rolling window and crossing logic

Processing is pure Python, independent of Kafka (`src/processor.py`), which makes the logic directly unit-testable. The window is event-time based and inclusive: an observation exactly 15 minutes old is still in the window; one second older is not. Duplicate `time_tags` and late (out-of-order) events are counted and skipped without touching window state — a skipped record can never create, suppress, or rearm an alert. The alert is edge-triggered on the rolling maximum with hysteresis-by-rearm: sustained elevation emits nothing new until the maximum has fallen below 6.

## Why one topic on one local Kafka broker

The planetary Kp series is a single global sequence, so one topic with one partition and a constant key is the design that preserves order without ceremony. Kafka still earns its place: it decouples the validated producer from the consumer, gives replayable offsets, and makes duplicate handling observable. A local Docker broker (official `apache/kafka` image, KRaft mode) keeps the demo reproducible for any reviewer; the demo script recreates broker state on every run precisely so no run can silently reuse stale topics or consumer offsets. The producer is idempotent with `acks=all`; the consumer uses a fresh group id per run, a bounded idle timeout, and skips malformed messages with a visible warning instead of crashing.

## 4. Evidence, bounded AI, and reproducibility

### Validation and evaluation results

- **Automated tests:** 167 standard tests passed (contract, processor, outputs, briefing, evaluation, demo fixtures, UI, and live poller). The 3 opt-in Kafka integration tests also passed separately against a clean Docker broker.
- **End-to-end acceptance:** `./run_demo.sh` verifies the generated artifacts field-by-field against the committed oracle `data/fixtures/replay_expected.json` — 9 messages consumed, 8

unique, 1 duplicate skipped, exactly 2 alerts at the labeled timestamps — then runs the full test suite; three consecutive runs passed identically, and a deliberately broken run exits nonzero.

- **Evaluation evidence:** `python -m src.evaluate` regenerates `evaluation/evaluation.json` deterministically (byte-identical across runs, no network, no key): 7/7 assertions — the replay alert comparison plus six fixed AI-briefing cases.

## Bounded AI and its verification

The language model (OpenAI `gpt-5-nano`, optional) sees only five already-calculated facts — latest Kp, rolling maximum, UTC window start/end, and the rule-based label — and may only phrase them as a two-sentence briefing. It cannot trigger, suppress, or relabel an alert. An automatic validator rejects any candidate containing a Kp number not in the input, a mismatched label, unsupported causes/locations/recommendations, more than two sentences, or empty output, and records the explicit rejection reason; a rejected candidate is never silently repaired. The six committed evaluation cases (correct, wrong number, wrong label, unsupported detail, too long, API unavailable) all produce their expected decisions. With no key, a failed request, or a rejected response, a deterministic two-sentence template built from the same facts writes `briefing.txt` — the fallback is the required path, not an error state. Full disclosure of runtime and development AI use is in `AI_USAGE.md`.

## Exact review path

```
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
./run_demo.sh           # requires Docker Desktop; after setup, no external
                        # network or OpenAI key
```

Expected artifacts: `outputs/alert.json` (two alerts), `outputs/metrics.csv` (nine rows), `outputs/briefing.txt` (two sentences), and `evaluation/evaluation.json` (7/7). The same command passes with `OPENAI_API_KEY` unset, with the briefing explicitly reported as the deterministic fallback. Cleanup: `docker compose down -v`.

## Team contributions

| Area                                                                                                                                                                                                                                 | Lead   | Cross-review |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|--------------|
| Event contract, Kafka I/O, replay producer, processor and crossing rule, alert/metrics writers, stream consumer, briefing + validator, evaluation harness                                                                            | Noam   | Niki         |
| Test-coverage audits and contract edge-case tests, one-command demo script ( <code>run_demo.sh</code> ), end-to-end and offline acceptance runs, README/data-source finalization, architecture chart, report and presentation drafts | Niki   | Noam         |
| Fixture design, source verification, checklist and workflow                                                                                                                                                                          | Shared | Both         |

All work merged through reviewed pull requests on task-scoped branches; both partners ran the full demo independently and can explain every stage.

### **Limitations and next step**

**Limitations.** The alert reports a recent estimated global Kp condition; it is not a forecast and not a station-level propagation model. NOAA's one-minute values are preliminary and revisable. The 15-minute window and Kp 6 threshold are a course-scale streaming rule, not an official alerting algorithm. The AI validator checks facts, sentence count, and banned unsupported details — it cannot prove a briefing is well-phrased, and a candidate could avoid banned terms yet still mislead, which is why rejection defaults to the deterministic fallback. The demo replays a labeled synthetic fixture because quiet live data cannot demonstrate the threshold crossing.

**One realistic next step.** Add a long-running consumer that maintains rolling-window state while the implemented 60-second NOAA poller continues publishing new timestamps. It would update the artifacts atomically and let the optional UI refresh automatically. The deterministic replay would remain the reviewer path because live conditions may never cross Kp 6 during a demonstration.